

01. Project Description

Dynamic Workload Allocation Between Space and Ground

Team [pas encore]

Version 2.0 - September 2026

Table des matières

1	Introduction and Motivation	2
2	The Research Objective	2
2.1	Theoretical Grounding : Fundamental Performance Bounds	2
3	The Four Allocation Strategies (Division of Labor)	3
3.1	Validation Strategy : From Controlled to Realistic	5
4	What We Measure (Overview Only)	5
5	Technology Stack (Rationale)	6

1 Introduction and Motivation

Welcome to the team! This project addresses a critical bottleneck in future orbital data centers : **the dynamic allocation of computational tasks** across a fleet of heterogeneous nodes (satellites, ground stations, and terrestrial clouds) when communication windows are intermittent and energy is scarce.

The core challenge (Problem 6, IASTAM 6.0) is to design a decision engine that dynamically assigns each incoming workload to the most relevant computing location based on the current context, while maintaining performance when the system state changes (e.g., link loss, CPU overload). This scheduling problem is a stochastic and dynamic variant of the Resource-Constrained Project Scheduling Problem (RCPSP), which is **NP-hard**; this way we may need madam rawya; an exact real-time solution therefore becomes impractical as the number of tasks and constraints grows.

To study this, we will implement and evaluate **five distinct allocation strategies** on a standardized discrete-event simulation environment. By holding the environment (topology, workload) constant, any observed difference in performance is attributable to the **orchestration/allocation strategy**, which is the object of our study.

2 The Research Objective

From a formal computational perspective, this problem is a stochastic, dynamic variant of the **Resource-Constrained Project Scheduling Problem (RCPSP)**. It is **NP-hard** in the strong sense, meaning that exact solutions (e.g., via Mixed-Integer Linear Programming) do not scale to real-time operations with more than a handful of tasks.

Therefore, we abandon the search for an optimal exact solution and focus on comparing **meta-heuristics and learning-based control policies**. We will implement five distinct allocation strategies, ranging from deterministic baselines to an intelligent agent, and evaluate all five on the same simulated environment.

The research question is :

How does a proactive, learning-based allocation strategy compare to reactive heuristics regarding the primary mission metrics? Which strategy best approximates the optimal Pareto frontier between execution time and energy efficiency?

2.1 Theoretical Grounding : Fundamental Performance Bounds

Our methodological approach is anchored in two established theoretical frameworks that justify the necessity of dynamic scheduling.

First, **Amdahl's Law :proposed by Yassine** provides an upper bound (limite supérieure we can't achieve 100 pour 100 fama un compromis) on achievable speedup. Since a fixed fraction of mission time is inherently consumed by communication latency and orbital visibility windows (the non-parallelizable fraction), improving computational speed alone yields diminishing returns. This explains why strategies that simply offload everything to the cloud (Strategy 1) inevitably hit a performance ceiling and justifies our focus on intelligent data reduction and proactive offloading.

Second, we model the infrastructure as a network of **M/G/1 queues**. In this model, the average waiting time increases nonlinearly with the utilization $\rho = \lambda/\mu$ of each node

and diverges as $\rho \rightarrow 1$, as described by the Pollaczek–Khinchine formula. Little’s Law then relates the average number of tasks in the system to their average waiting time. Our orchestrator’s primary objective is therefore to prevent ρ from approaching 1 on any single satellite. The Reinforcement Learning agent (Strategy 5) is specifically trained to minimize sustained high utilization, with the goal of maintaining bounded latency even under link failures.

3 The Four Allocation Strategies (Division of Labor)

Each team member (or pair) implements a family of strategies. The grouping is based on complexity, from static to fully adaptive.

Team 1 : The Static Baselines

Objective : Provide lower-bound performance benchmarks that cost nothing to compute.

Strategy 1. Always Ground (A-Ground) A deterministic, zero-intelligence strategy. All incoming tasks are immediately routed to the terrestrial cloud (or ground station). *Use case* : Represents the "traditional" approach when no onboard compute exists.

Strategy 2. Always Orbital (A-Orbital) A deterministic, zero-intelligence strategy. All incoming tasks are executed on the local satellite (LEO node). *Use case* : Represents the "edge-only" approach, minimizing transmission latency but risking resource exhaustion.

Remarque : ces deux strategy sont essentiellement developper pour servir par la suite ,dans l’évaluation, comme reference.

Team 1 (Continued) : The Adaptive Algorithms

Objective : Implement dynamic decision-making that reacts to the environment.

Strategy 3. Greedy Heuristic (Score-based) A reactive orchestrator that calculates a dynamic "suitability score" for each available node for each incoming task. The score considers current CPU load, estimated energy consumption, and link latency. The task is assigned to the node with the best instantaneous score. *Use case* : Fast, low-overhead decision-making based on the current system snapshot.

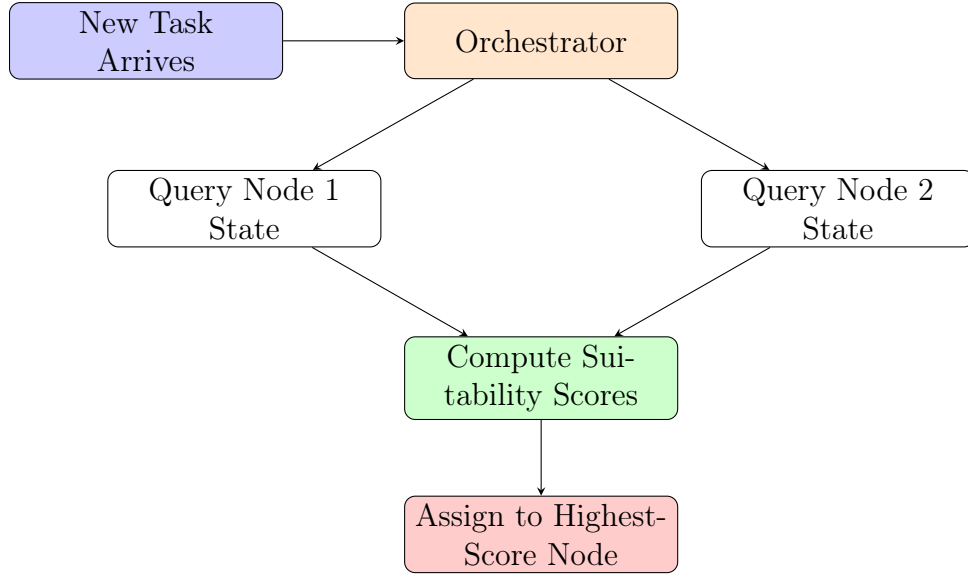


FIGURE 1 – Strategy 3 : Greedy Heuristic Flow

Strategy 4. Dynamic Resilient Scheduler (Proactive Re-allocation) A proactive orchestrator that not only assigns tasks based on scores but also includes a monitoring daemon. If a node fails or a link drops, it automatically triggers a **re-scheduling** event, migrating mid-flight or pending tasks to the next best available node without human intervention.

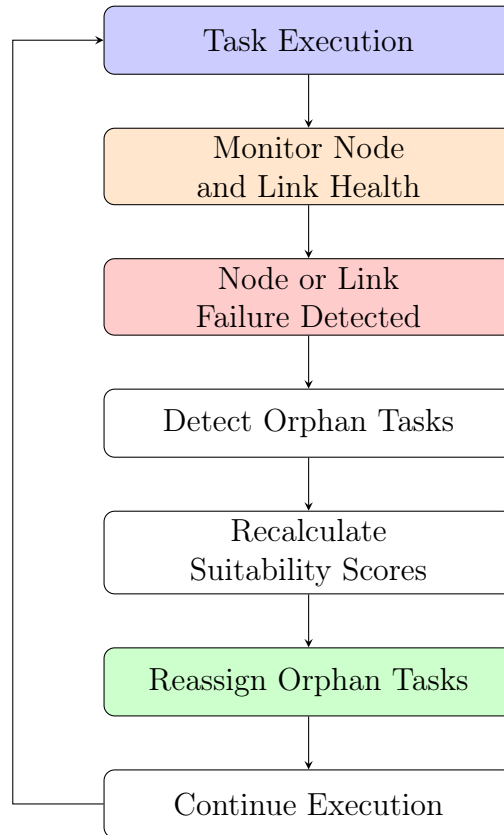


FIGURE 2 – Strategy 4 : Dynamic Resilient Flow

Team 2 : The Artificial Intelligence Approach

Strategy 5. Deep Reinforcement Learning Agent (DQN / PPO)(which i recently begin to learn it) Instead of relying on human-coded rules (Strategy 4), we implement an autonomous agent that learns an optimal scheduling policy through interaction with the simulation environment. The agent observes the system state (CPU load, battery levels, link status, queue length) and outputs a probability distribution over the available nodes.

- **Algorithm** : Proximal Policy Optimization (PPO) or DQN, implemented via `stable-baselines3`.
- **Reward Function** : $R = -\alpha \cdot M_T - \beta \cdot M_E + \gamma \cdot \mathbf{1}_{\text{DeadlineMet}}$.
- **Goal** : Learn to anticipate network disconnections and energy shortages, unlike the myopic Greedy heuristic.

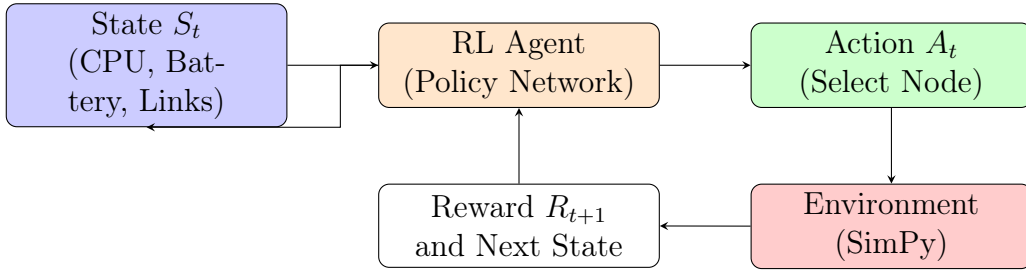


FIGURE 3 – Strategy 5 : Closed-loop Reinforcement Learning for dynamic scheduling.

Note on numbering : strategies are numbered 1 to 5 in this order. Strategies 1–4 are implemented by Team 1; Strategy 5 is implemented by Team 2 as the high-risk, high-reward research contribution.

3.1 Validation Strategy : From Controlled to Realistic

To avoid the pitfall of testing on a single, uninterpretable mega-scenario, we adopt a two-step validation approach.

Step 1 (Primary, Phase 2 & 3) : We rely on **four synthetic workload profiles** (Burst Heavy, Energy Critical, Radiation-Induced Failures, Mixed Priority). Each profile is designed to independently stress exactly one of our target metrics (M_T , M_E , M_R , M_L). This guarantees that if a strategy fails, we can precisely diagnose the cause (e.g., "the RL agent fails because it does not handle memory errors, not because of load spikes").

Step 2 (Secondary, Phase 3) : To demonstrate operational realism, we will optionally validate the best-performing strategy on a **merged scenario** combining :

- Task dependencies from **PSPLIB** benchmarks.
- Arrival timestamps from **Google cluster traces**.
- Link availability computed via **Skyfield** orbital mechanics.

This ensures our findings are both *causally interpretable* (Step 1) and *operationally realistic* (Step 2).

4 What We Measure (Overview Only)

For each strategy and each workload profile, the simulator emits telemetry from which we compute the four core metrics defined in `03_EVALUATION_METRICS_SPECIFICATION`.

md. At a glance :

TABLE 1 – Core Metrics Intuition

Metric	Intuition
M_T (Processing Time)	Total makespan. How long does it take to finish the entire batch of tasks?
M_L (Load Balance)	Fairness. Are we overloading one satellite while others are idle?
M_R (Resilience)	Robustness. How many tasks are successfully recovered after a simulated failure?
M_E (Efficiency)	Resource utilization. How much energy/bandwidth is consumed per completed task?

This table is intentionally informal. The only canonical definitions, formulas, and normalizations live in `03_EVALUATION_METRICS_SPECIFICATION.md`. When implementing, code against that document.

5 Technology Stack (Rationale)

Implementation : Pure Python + SimPy + Pydantic

Why are we NOT using real satellite hardware or complex distributed orchestration tools?

The challenge requires a **demonstrator** that proves the concept works. Real hardware is inaccessible. Existing cloud orchestration tools (e.g., Kubernetes) assume persistent connectivity and ignore energy constraints.

We implement a **Discrete-Event Simulation** using **SimPy**. This allows us to simulate complex temporal dynamics (e.g., 10-minute visibility windows, variable task processing times) deterministically and reproducibly. **Pydantic** enforces strict state management for all nodes, tasks, and links, ensuring every decision is fully auditable. This guarantees we can replay any scenario and prove the "resilience" of Strategy 4 by injecting failures at specific timestamps.

Environment, installs, and the starter template are in `04_ONBOARDING_AND_SETUP.md`. Simulation constraints and workload profiles are in `02_EXPERIMENTAL_DESIGN.md`.